



Bacharelado em Ciência da Computação

Componente: Computação Gráfica → Professor: Claudio Kleina

Aula 3 – Introdução ao Universo 3D com OpenGL

1 O Espaço Tridimensional (XYZ)

Até agora, trabalhamos com o plano cartesiano bidimensional, onde tínhamos apenas **Largura (X)** e **Altura (Y)**. No 3D, introduzimos a **Profundidade (Z)**.

No OpenGL, por padrão, o sistema de coordenadas funciona da seguinte forma:

- **Eixo X:** Horizontal (Negativo para a esquerda, Positivo para a direita).
- **Eixo Y:** Vertical (Negativo para baixo, Positivo para cima).
- **Eixo Z:** Profundidade (**Negativo "entra"** na tela, **Positivo "sai"** da tela em sua direção).

2 O Pipeline Gráfico e a Câmera

Imagine que o OpenGL é uma câmera fotográfica digital. Para tirar uma foto de um objeto 3D e exibi-la em sua tela (que é 2D), o computador realiza um processo chamado **Projeção**.

- **Projeção Perspectiva:** É como o olho humano enxerga. Objetos que estão mais longe (com valor de Z mais negativo) parecem menores do que objetos que estão perto. É ideal para jogos e simulações realistas.
- **Projeção Ortográfica:** Não há distorção de tamanho baseada na distância. É usada em jogos 2D, plantas arquitetônicas ou interfaces (HUD).

3 O Problema da Visibilidade: Z-Buffer

Em um desenho 2D, o último objeto a ser desenhado fica "por cima" dos outros. No 3D, isso não funciona. Um objeto que está em $Z = -5$ deve ser escondido por um objeto que está em $Z = -2$, independentemente da ordem em que foram codificados.

Para resolver isso, usamos o **Z-Buffer (ou Depth Buffer)**. Ele é uma memória que guarda a distância de cada pixel até a câmera. Antes de desenhar um novo pixel, o OpenGL verifica: "O pixel que já está lá é mais perto do que o que eu quero desenhar agora?". Se for, ele descarta o pixel do novo desenho.

4 Anatomia do Código Inicial

Para iniciar qualquer projeto 3D, precisamos habilitar essas funcionalidades:

- **glEnable(GL_DEPTH_TEST):** Ativa o teste de profundidade para que objetos distantes não apareçam na frente de objetos próximos.
- **glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT):** No início de cada quadro (frame), limpamos não apenas as cores da tela anterior, mas também as informações de profundidade.

5 Primitivas: O Triângulo é o Rei

No 3D, quase tudo é feito de triângulos. Por quê? Porque três pontos no espaço sempre formam um plano perfeito. Quadrados ou polígonos complexos podem sofrer distorções se um dos vértices for movido para fora do plano, mas um triângulo nunca "dobra".

Exer

cício de Reflexão

Se

um objeto A está na posição (0, 0, -10) e um objeto B está na posição (0, 0, -5), qual deles parecerá maior na tela sob uma projeção perspectiva?

1. O que aconteceria com o seu jogo se você esquecesse de usar o comando `glClear(GL_DEPTH_BUFFER_BIT)`? (Dica: tente comentar essa linha no seu código e observe o resultado).

Construindo e Movimentando Objetos no Espaço

1 Modelagem Manual: Vértices e Faces

No OpenGL, não "carregamos" um cubo pronto com um comando simples; nós o construímos. Um cubo possui **8 vértices** (cantos) e **6 faces** (lados).

Para desenhar uma face quadrada, usamos a primitiva `GL_QUADS`. A ordem em que você define os vértices é crucial: eles devem ser listados em sentido **anti-horário** para que o OpenGL entenda para qual lado a "frente" da face está apontando.

Exemplo de coordenadas para uma face frontal:

- Inferior Esquerdo: (-1, -1, 1)
- Inferior Direito: (1, -1, 1)
- Superior Direito: (1, 1, 1)
- Superior Esquerdo: (-1, 1, 1)

2 Transformações Geométricas

Uma vez que o objeto está definido, não mudamos os valores dos seus vértices para movê-lo. Em vez disso, transformamos o **espaço** ao redor dele. Existem três transformações fundamentais:

- **Translação** (`glTranslatef(x, y, z)`): Move o objeto. Se você quer que o cubo apareça à direita, usa valores positivos de X. Se quer que ele se afaste de você, usa valores negativos de Z.
- **Rotação** (`glRotatef(ângulo, x, y, z)`): Gira o objeto. O primeiro parâmetro é o ângulo em graus. Os outros três definem sobre qual eixo o giro ocorre (ex: 1, 0, 0 para girar como um rolo compressor).
- **Escala** (`glScalef(x, y, z)`): Altera o tamanho. Um valor de 2.0 dobra o tamanho, enquanto 0.5 o reduz pela metade.

3 A Importância da Ordem das Operações

A ordem em que você escreve as funções de transformação altera o resultado final:

1. **Girar e depois Mover:** O objeto gira em torno do próprio centro e depois vai para o lado.
2. **Mover e depois Girar:** O objeto orbita o centro da cena (como um planeta ao redor do Sol).

4. Matrizes e o `glLoadIdentity()`

O OpenGL é uma "máquina de estados". Se você transladar um objeto 5 unidades para a direita no Frame 1, e não resetar o sistema, no Frame 2 ele estará a 10 unidades.

O comando `glLoadIdentity()` serve para "limpar" as transformações e voltar a câmera para a origem \$(0,0,0)\$, garantindo que cada novo desenho comece do zero.

Guia Prático: Criando Sólidos

Para que um objeto pareça sólido e não um emaranhado de linhas, seguimos estes passos no código:

1. **Limpeza:** Limpar buffers de cor e profundidade.
2. **Reset:** `glLoadIdentity()` para garantir que transformações anteriores não afetem o objeto atual.
3. **Posicionamento:** Aplicar `glTranslatef` para colocar o objeto no campo de visão da câmera (geralmente recuando no eixo Z).
4. **Animação:** Aplicar `glRotatef` usando uma variável de tempo ou contador.
5. **Desenho:** Abrir o bloco `glBegin(GL_QUADS)`, definir cores e vértices, e fechar com `glEnd()`.

Controle pelo teclado com GLFW: Verificação de Estado (Polling)

Esta é a forma mais comum para movimentação de personagens. Você pergunta ao GLFW, dentro do loop principal, se uma tecla está pressionada naquele exato momento.

Ideal para: Movimentação (frente, trás, rotação).

Exemplo:

```
pos_x, pos_y = 0.0, 0.0 # Globais
```

```
# Dentro do loop:
```

```
if glfw.get_key(janela, glfw.KEY_LEFT) == glfw.PRESS: pos_x -= 0.05
```

```
if glfw.get_key(janela, glfw.KEY_RIGHT) == glfw.PRESS: pos_x += 0.05
```

```
# No desenho:
```

glTranslatef(pos_x, pos_y, -5)

- **Vantagem:** Resposta imediata e processamento a cada frame.
- **Desvantagem:** Se você quiser apenas "trocar uma opção", uma pressão rápida pode ser lida várias vezes em um segundo (devido ao FPS alto), causando instabilidade.

1. Teclas Alfanuméricas

Seguem o padrão da letra ou número correspondente:

- glfw.KEY_A até glfw.KEY_Z
- glfw.KEY_0 até glfw.KEY_9

2. Teclas de Direção (Setas)

Muito utilizadas para movimentação de câmera e objetos:

- glfw.KEY_UP: Seta para Cima
- glfw.KEY_DOWN: Seta para Baixo
- glfw.KEY_LEFT: Seta para Esquerda
- glfw.KEY_RIGHT: Seta para Direita

3. Teclas de Modificação

Geralmente usadas em combinação com outras teclas (atalhos):

- glfw.KEY_LEFT_SHIFT / glfw.KEY_RIGHT_SHIFT
- glfw.KEY_LEFT_CONTROL / glfw.KEY_RIGHT_CONTROL
- glfw.KEY_LEFT_ALT / glfw.KEY_RIGHT_ALT
- glfw.KEY_CAPS_LOCK

4. Teclas de Função e Controle

- glfw.KEY_SPACE: Barra de Espaço
- glfw.KEY_ENTER: Tecla Enter
- glfw.KEY_ESCAPE: Tecla Esc
- glfw.KEY_TAB: Tecla Tab
- glfw.KEY_BACKSPACE: Apagar
- glfw.KEY_F1 até glfw.KEY_F12

5. Teclas de Navegação e Edição

- glfw.KEY_INSERT
- glfw.KEY_DELETE
- glfw.KEY_PAGE_UP
- glfw.KEY_PAGE_DOWN
- glfw.KEY_HOME
- glfw.KEY_END

6. Teclado Numérico (Keypad)

- glfw.KEY_KP_0 até glfw.KEY_KP_9
- glfw.KEY_KP_ADD: Sinal de +
- glfw.KEY_KP_SUBTRACT: Sinal de -
- glfw.KEY_KP_MULTIPLY: Sinal de *
- glfw.KEY_KP_DIVIDE: Sinal de /
- glfw.KEY_KP_ENTER: Enter do teclado numérico

##Código para as aulas comentado

import glfw #Graphics Library Framework

#Gerencia a janela e eventos (teclado/mouse)

Mais atual que o GLUT // O SDL é mais indicado para jogos / multimídia

from OpenGL.GL import * # Funções principais de desenho e estados do OpenGL

from OpenGL.GLU import * # Funções utilitárias para matrizes de câmera e projeção

def desenhar_cubo():

"" Define a geometria e as cores do cubo face por face ""

glBegin(GL_QUADS) # Instrução para o OpenGL agrupar cada 4 vértices em um quadrado

--- Face Frontal (Vermelha) ---

glColor3f(1, 0, 0) # Define a cor do "pincel" (R, G, B)

glVertex3f(-1, -1, 1); glVertex3f(1, -1, 1); glVertex3f(1, 1, 1); glVertex3f(-1, 1, 1)

--- Face Traseira (Verde) ---

glColor3f(0, 1, 0)

glVertex3f(-1, -1, -1); glVertex3f(-1, 1, -1); glVertex3f(1, 1, -1); glVertex3f(1, -1, -1)

--- Face Superior (Azul) ---

glColor3f(0, 0, 1)

glVertex3f(-1, 1, -1); glVertex3f(-1, 1, 1); glVertex3f(1, 1, 1); glVertex3f(1, 1, -1)

--- Face Inferior (Amarela) ---

glColor3f(1, 1, 0)

glVertex3f(-1, -1, -1); glVertex3f(1, -1, -1); glVertex3f(1, -1, 1); glVertex3f(-1, -1, 1)

--- Face Direita (Magenta) ---

glColor3f(1, 0, 1)

glVertex3f(1, -1, -1); glVertex3f(1, 1, -1); glVertex3f(1, 1, 1); glVertex3f(1, -1, 1)

--- Face Esquerda (Ciano) ---

glColor3f(0, 1, 1)

glVertex3f(-1, -1, -1); glVertex3f(-1, -1, 1); glVertex3f(-1, 1, 1); glVertex3f(-1, 1, -1)

glEnd() # Finaliza o envio de vértices para a placa de vídeo

def main():

Inicializa o framework GLFW

if not glfw.init():

return

Cria a janela onde o conteúdo será renderizado

janela = glfw.create_window(800, 600, "Aprendendo OpenGL 3D", None, None)

if not janela:

glfw.terminate()

return

Ativa o contexto OpenGL nesta janela específica

glfw.make_context_current(janela)

Ativa o Z-Buffer (Depth Test). Sem isso, as faces de trás seriam desenhadas

por cima das da frente se fossem processadas por último no código.

glEnable(GL_DEPTH_TEST)

--- Configuração da Lente (Projeção) ---

glMatrixMode(GL_PROJECTION) # Entra no modo de edição da "lente da câmera"

glLoadIdentity() # Limpa qualquer configuração anterior

Parâmetros: (Campo de visão em graus, Proporção da tela, Distância mínima, Distância máxima)

gluPerspective(45, 800/600, 0.1, 50.0)

```

# --- Configuração do Mundo (Modelview) ---
glMatrixMode(GL_MODELVIEW) # Entra no modo de posicionar objetos e a própria câmera

# Loop que mantém o programa rodando até fechar a janela
while not glfw.window_should_close(janela):
    # Limpa a tela com a cor de fundo (preto por padrão) e limpa o buffer de profundidade
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # Reseta a matriz de transformação para o centro (0,0,0)
    glLoadIdentity()

    # Move o objeto 5 unidades para "dentro" da tela (afastando-o de nós)
    # O OpenGL usa um sistema de mão direita, então -Z entra na tela.
    glTranslatef(0, 0, -5) # testar com -51

    # Rotaciona o espaço. Parâmetros: (ângulo em graus, eixo_x, eixo_y, eixo_z)
    # Aqui ele gira 50 graus por segundo nos eixos X e Y simultaneamente.
    glRotatef(glfw.get_time() * 50, 1, 1, 0)

    # Executa os comandos de desenho definidos na função lá em cima
    desenhar_cubo()

    # Inverte o buffer de desenho com o buffer de exibição (Double Buffering)
    # Isso evita que vejamos a imagem sendo "escaneada" ou piscando.
    glfw.swap_buffers(janela)

    # Verifica se houve eventos (teclado, mouse, redimensionamento)
    glfw.poll_events()

# Finaliza e limpa os recursos do sistema
glfw.terminate()

# Execução do programa
if __name__ == "__main__":
    main()

```

Exercícios Sugeridos

1. **O Desafio do Quadrado Perdido:** Tente desenhar um cubo, mas deixe uma das faces sem cor. O que acontece quando você gira o cubo e olha para o "buraco"?
2. **Paralelepípedo:** Use a função `glScalef(2.0, 1.0, 0.5)` antes de desenhar o seu cubo. Qual foi a face que ficou mais comprida?
3. **Rotação Combinada:** Tente girar o objeto em dois eixos ao mesmo tempo: `glRotatef(ângulo, 1, 1, 0)`. Como isso altera a percepção de 3D comparado a girar em apenas um eixo?
4. Modifique o código para que o cubo se transforme em um paralelepípedo (uma barra longa). Ele deve ter o triplo da largura (eixo X) e metade da espessura (eixo Z).
Dica: Utilize a função `glScalef(x, y, z)` antes de chamar `desenhar_cubo()`.
5. Crie variáveis globais `pos_x` e `pos_y`. Utilize as teclas de seta do teclado para mover o cubo para a esquerda, direita, cima e baixo na tela.
Desafio: Não permita que o cubo saia dos limites visíveis da janela.
6. **Câmera Dinâmica (Campo de Visão):** Vincule as teclas W e S para alterar o valor do **FOV** (Field of View) na função `gluPerspective`.

Pergunta: O que acontece com a distorção dos objetos quando o FOV é muito alto (ex: 120°) versus quando é muito baixo (ex: 10°)?

7. Comente a linha `glEnable(GL_DEPTH_TEST)` e execute o programa.
8. Descreva o que aconteceu visualmente quando o cubo rotaciona. Por que as faces de trás parecem estar na frente? Por que o comando `glClear(GL_DEPTH_BUFFER_BIT)` deixa de fazer sentido nesse cenário?

Bom estudo!